Protocol Extension and Runtime Composition: Schema Design, the Coordinator Pattern, and the Software Architecture Above the Kernel

Alessandro Daliana

May 2026

Abstract

The FIGARO kernel is a settlement primitive: two operations (`commit` and `resolveProcess`), three storage mappings, no upgrade path, no escape hatches. Its mechanism-design derivation (asymmetric bonding, buyer dominance with atomic resolution, and the escape-hatch-weakness theorem) and its multi-method formal verification (TLA$^+$, symbolic execution, property-based fuzzing, and Certora specifications discharging the kernel's invariants) are treated separately and stay strictly at the kernel tier.

This paper covers what stands above the kernel as a research object. The kernel by itself is not a useful protocol; it is a settlement primitive that becomes a protocol when extensions are composed onto it. Three discipline questions arise. First, when should a protocol extension be written, and what does it mean for an extension to preserve the kernel's equilibrium properties? We treat the *protocol extension doctrine* as the answer: the anchored artifact pattern, append-only identity, first-write-wins binding, the boundary between per-instance payloads and shared reference semantics. Second, how should new schemas be designed and verified at the protocol layer? We treat *schema design as a computer-science discipline*: the four-layer verification stack (Layer A specification, Layer B prover mirror [pending], Layer C Solidity validator, runtime-tier UI binding), the 18-schema catalogue as a coordinated reference set, and the `SchemaRegistrationHelper` atomic-bind pattern that closes the front-running window between schema registration and validator binding. Third, how does the *coordinator pattern* preserve the kernel's bonding equilibrium when an external mechanism is composed onto the kernel? We give a formal definition of composition semantics, state sufficient conditions for equilibrium preservation, and treat the `AttestationCoordinator` as the worked verification example where the parametric Core-immutability rules in Certora discharge the conditions. Finally, we describe the *runtime composition* above the protocol tier: the seven-layer composition pipeline (protocol truth, semantic derivation, institution assembly, mechanism packages, service bindings, view definitions, skin bundles), the assembly registry pattern, the module system, the designer canvas, and the `(marketing)`/`(app)` route-architecture split.

The paper is in computer-science register: software architecture as a research object, with the discipline derived from the kernel's security requirements rather than from convenience.

Keywords: protocol extension, schema design, coordinator pattern, equilibrium-preserving composition, runtime architecture, software composition, content-addressed identity

1 Introduction

The FIGARO kernel is a settlement primitive whose two operations, three storage mappings, and frozen invariants produce a Nash equilibrium under asymmetric bonding rules and buyer dominance with atomic resolution. As deployed, the kernel satisfies those invariants under multiple formal-verification methods. The kernel itself stops at its boundary because that is where the formal results live.

The kernel by itself is not a useful protocol. A settlement primitive needs a graph above it that participants can compose: schemas typing the agreement content, mechanism contracts coordinating specific patterns, runtime surfaces letting humans and agents interact with the resulting institutional shapes. The graph is what makes the kernel applied; the kernel is what makes the graph trustworthy. The two are inseparable in any working deployment, and the discipline that governs the second half — how protocol extensions are written and how runtime compositions are organized — is the subject of the present paper.

The argument is in three parts.

The protocol extension doctrine. Extensions to the kernel must preserve the kernel's equilibrium properties or they break the protocol. The discipline is therefore not a matter of taste: there is a decision rule for whether a new domain feature belongs in the protocol or stays in app logic, and the rule is checkable. We state the rule, develop the anchored artifact pattern as the recurring structural shape, and identify the design guardrails the rule produces.

Schema design as a computer-science discipline. A FIGARO schema is not a JSON document on a filesystem. It is a typed-data definition deployed across four coordinated layers (specification, prover mirror, on-chain validator, runtime-tier binding) under append-only identity discipline, with first-write-wins binding to the validator contract and content-addressed reference to the specification document. The discipline is a computer-science research object, not just an implementation detail, and we treat it as such. The 18-schema catalogue currently shipped in the protocol is the worked-example reference set.

The coordinator pattern formally. The *coordinator pattern* is the discipline under which an external mechanism contract composes with the kernel without breaking the bonding equilibrium: an extension contract satisfies specified conditions on its read/write profile against kernel state. We state the conditions formally with composition semantics and prove that the conditions are sufficient. The `AttestationCoordinator` contract is the worked verification example, with the parametric Core-immutability rules in `certora/AttestationCoordinator.spec` discharging the sufficient conditions.

Runtime composition. Above the protocol tier sits the runtime tier, where institutional assemblies, mechanism packages, service bindings, view definitions, and skin bundles compose into rendered institutions that humans and agents interact with. We describe the seven-layer composition pipeline, the assembly registry pattern, the module system, the designer canvas, and the architectural separation between marketing surfaces and operational surfaces. The runtime is software architecture, not user-experience design; we treat it as a CS object.

Paper organization. Section 2 summarizes the settlement primitive in the register the present paper uses. Section 3 develops the protocol extension doctrine. Section 4 treats schema design as a CS discipline. Section 5 formalizes the coordinator pattern. Section 6 describes runtime composition. Section 7 catalogues what the discipline refuses. Section 8 concludes.

2 The Settlement Primitive

The bonded primitive's mechanism-design derivation and its formal verification are out of scope for the present paper. We summarize the kernel features the present paper relies on.

Kernel surface. Two state-changing operations: `commit` (dual-signed EIP-712 commitment, locks asymmetric bonds) and `resolveProcess` (buyer-only, atomic settlement of all orders in a process). Three storage mappings: `processes` (`ProcessState`), `orderStatus` (`uint8`), `orderProcessId` (`bytes32`).

Six invariants. (i) asymmetric bonding (buyer locks $2P$, seller locks $2G$); (ii) progressive collateralization across N-party process chains; (iii) buyer dominance (only the root buyer can resolve); (iv) atomic resolution (all active orders settle simultaneously or not at all); (v) immutable evidence (commits and resolutions emit unmodifiable events); (vi) no escape hatches (no admin, no timeout, no governance, no unilateral exit).

Three tiers. *Kernel*: the irreducible settlement primitive. *Protocol*: the kernel together with extensions composed under the discipline of this paper. *Runtime*: the protocol together with semantic layers, builder surfaces, and rendered institutional shapes that humans and agents interact with. The present paper takes the three-tier distinction as given and develops the protocol and runtime tiers.

3 The Protocol Extension Doctrine

The kernel is intentionally narrow. Its narrowness is what produces the bonding equilibrium; widening it would weaken the equilibrium. Extensions to the kernel therefore must add capability without widening the kernel itself. The doctrine answers the question of how this is done.

3.1 What the kernel secures

At the kernel layer the protocol secures five things and only these: (i) process topology (which orders compose into which processes); (ii) economic obligations between counterparties (the bond posture at each order); (iii) role-bearing order nodes (who is the buyer and who is the seller at each order); (iv) lifecycle and settlement history (when each order was committed and resolved); (v) atomic process resolution semantics (the all-or-nothing settlement rule).

The kernel does not encode domain-specific meaning. A bonded commitment between a passenger and an airline, between a shipper and a forwarder, or between an agent and a service provider all reduce to the same kernel objects (an order with a seller, a buyer, a payment, and an agreement-hash). Domain meaning is added by extensions that attach typed information to the secured process graph.

3.2 The anchored artifact pattern

The recurring structural shape of a FIGARO extension is the *anchored artifact pattern*. An anchored artifact family exists when:

(1) multiple parties or tools must share a stable interpretation of some referenced artifact;

(2) that interpretation must remain auditable over time;

(3) the protocol needs a minimal on-chain reference point for that artifact family.

The pattern then is: off-chain semantics (the document, the methodology, the field catalogue, the legal interpretation) plus an on-chain anchor for shared reference integrity. The anchor carries only what the chain needs to know to identify the artifact, verify that the off-chain document referenced is the one that was committed, and check that the artifact was admitted for use at the time of reference. The anchor does not carry the semantics themselves.

Definition 3.1 (Anchor record)**.** A protocol-layer anchor for an artifact family is a tuple $\langle \text{schemaId}, \text{version}, \text{uriHash}, \text{admitted} \rangle$ where schemaId is the anchor identity, version is the version within the family, uriHash is an immutable cryptographic hash of the URI pointing at the off-chain content, and admitted is the binding's monotone admission flag (false until first registration, true thereafter; the discipline forbids deactivation).[1]

The minimal-anchor surface is intentional. Larger anchors would freeze interpretive commitments into brittle on-chain state; smaller anchors would lose reference integrity. The four-field shape is the narrowest surface that supports cross-party agreement on which artifact is being referenced and which version of it.

3.3 Per-instance payloads versus shared reference semantics

The discipline turns on a distinction the doctrine makes load-bearing: the difference between *per-instance payloads* (the data attached to a particular order or process instance) and *shared reference semantics* (definitions whose meaning must remain stable across parties, tools, or time).

Per-instance payloads are operational data values: a specific delivery manifest, sealed address data, notes for a particular fulfilment event. These are typically private, mutable at the business level, or specific to one workflow instance. They do not deserve a protocol-level anchor. They live as instance data on the order or process that carries them.

Shared reference semantics are definitions whose interpretation must remain stable across counterparties, tools, or time: a disclosure schema, a manifest schema family, a certification framework reference, a quality-assurance reference standard. These may justify a protocol-level anchor under the anchored artifact pattern.

3.4 The decision rule

Proposition 3.2 (Decision rule for protocol extension)**.** *A new domain feature belongs in the protocol if and only if all three of the following hold:*

(1) the feature attaches meaning to the secured process graph rather than to private instance data;

(2) the feature requires stable shared interpretation across counterparties or tools;

(3) the protocol needs to preserve that reference integrity over time.

A feature satisfying all three conditions is a candidate for the anchored artifact pattern. A feature failing any of the three belongs in app logic, off-chain infrastructure, or per-instance payload handling.

The proposition is operationally checkable: each condition has a yes/no answer for a given proposed feature, and a feature that fails the check is not a protocol-extension candidate. The doctrine treats the rule as binding: the discipline is the willingness to keep features out of the protocol when the rule fails them, not just the willingness to add features when the rule passes them.

[1] In the deployed implementation (`src/SchemaRegistry.sol`), the admission flag is stored as `mapping(bytes32 ⇒ bool) public registered`; `version` and `uriHash` live on the `SchemaRegistered` event log rather than as queryable storage. The conceptual anchor is reconstructed by reading the event log under the append-only-identity discipline of Section 4.2.

3.5 Design guardrails

Four guardrails operationalize the rule:

1. Do not push app-specific workflow logic into the kernel. App logic that belongs to one institution does not belong to the protocol, regardless of how compelling the institution is.

2. Do not make the protocol so abstract that every document becomes a first-class ontology object. Bounded generality is the target; universal-ontology generality is a failure mode.

3. Do preserve reusable patterns when multiple domains clearly need the same coordination primitive. Generality earned by multiple domain instances is the kind of generality the protocol should accommodate.

4. Keep the protocol legible: process graph first, extension semantics second, app UX last. The order of legibility reflects the order of trust dependency.

The guardrails do not contradict the decision rule; they are the operational restatement of it. A proposed extension that fails any guardrail typically also fails the decision rule, and vice versa.

3.6 Bounded generality

The discipline's central tension is between under- and over-generality. Under-generality produces one-off app-specific modules that cannot become reusable protocol concepts; the protocol fragments into incompatible bilateral arrangements. Over-generality produces a fake universal ontology, abstract registries disconnected from concrete coordination problems, and protocol bloat that mistakes possibility for scope; the protocol becomes a CS-academic exercise rather than working coordination infrastructure.

The right level of generality is generic enough to support reusable extension patterns but concrete enough to stay grounded in process coordination, obligations, and verifiable reference integrity. The test for this balance is not theoretical: a proposed extension is the right level of generality if at least two distinct domains have plausibly demonstrated need for the same coordination primitive, and the primitive can be specified narrowly enough to support both without becoming a universal-ontology object.

4 Schema Design as a CS Discipline

A Figaro schema is the concrete realization of an anchored artifact family. Designing a schema is the operational form of applying the extension doctrine. We treat the discipline in computer-science register: typed-data design, content-addressed extension, verification-stack architecture.

4.1 The four-layer verification stack

A protocol-tier schema is deployed across four coordinated layers that together discharge the schema's correctness:

Layer A: client-side specification. A canonical JSON specification of the schema, parsed by a TypeScript meta-validator (a closed subset of JSON Schema covering string with specified format, integer, bigint as decimal string, boolean, enum, array, and object types with strict closed-schema rejection of unknown fields). The Layer A specification is the canonical declaration; everything below it is derived from it. Per-stage overrides express the staged-attestation

pattern (an `applied`-stage attestation may carry different fields than an `inspected_intact`-stage attestation against the same schema). The specification also carries a `categories` array of short tags (`handoff`, `commerce`, `emissions`, `lifecycle`, `jurisdiction`, `evidence-law`, etc.) that runtime UIs use for discoverability and grouping; the tags are non-normative for validation but form the discoverability surface for a schema's runtime presentation.

Layer B: prover mirror (pending). A Rust implementation of the Layer A validator running in the SP1 zero-knowledge prover that produces verifiable attestation receipts during batched off-chain execution. The prover guest program mirrors the TypeScript validator byte-for-byte to enforce the schema during batched attestation execution. Layer B is currently deferred; the TypeScript validator (Layer A) plus the on-chain validator (Layer C) serve as the conformance specification for the eventual Rust port.

Layer C: on-chain validator. A Solidity contract implementing `ISchemaValidator` that ABI-decodes the schema's content (no on-chain JSON parsing; the on-chain layer reads the canonical ABI encoding produced by Layer A's encoder) and reverts with typed custom errors on any violation. The validator is pure (no external state reads, no `block.*` or `tx.*`, no external calls) and binds to one schemaId via a compile-time literal (`bytes32 public constant override schemaId = keccak256("...")`); using a constructor-set `immutable` schemaId would forfeit the EVM-enforced determinism guarantee and is forbidden by the validator-contract pattern.

Runtime-tier UI binding. A frontend integration that consumes the Layer A specification via `useSchemaValidator(schemaId)`, renders schema-typed forms through the existing module and view system, and delivers the ABI-encoded content to the chain through the Layer C validator. The runtime tier is not strictly part of the verification stack, but it is part of the schema's deployment surface; an unintegrated schema exists on chain and not in any product.

Lockstep contract. A new schema is not done until all four layers ship in lockstep. If Layer A says one thing and Layer C says another, attestations silently divide into two interpretations and the schema's reference integrity collapses. The lockstep contract is operational discipline, not a formal soundness theorem: Layer A and Layer C agreement is enforced by the test suite and the deploy-script structure rather than proven by the verification stack. The lockstep contract is enforced at the deploy-script level (the schema's registration script registers the `schemaId` and binds the validator atomically) and at the test-suite level (Foundry tests in `test/schemaValidators/` cover well-formed input plus every typed-error revert path).

4.2 Append-only identity

Schema identity is append-only. The discipline produces three operational rules:

1. No in-place rewriting of schema meaning. A registered schema cannot be modified through any contract operation; the schema record at schemaId is immutable from the moment of registration.

2. No silent mutation of the content reference behind an existing identity. The `uriHash` field of the anchor is immutable; off-chain documents may be mirrored for availability but the canonical reference is the hash, not any URI.

3. New meaning requires a new version or a new schema identity. A schema family that needs a substantive change registers a new `schemaId` (e.g., `figaro-foo-v1` → `figaro-foo-v2`); never mutates the v1 contract or its Layer A spec.

The append-only rule preserves historical interpretability: attestations filed under schemaId_1 remain readable against the exact schema anchor they were filed under, regardless of subsequent schema-family evolution.

4.3 First-write-wins binding

The validator-contract binding to a schemaId is permissionless and first-write-wins. `AttestationCoordinator.s` `validator)` is callable by any address and binds the validator permanently for that schemaId. Subsequent calls revert with `ValidatorAlreadySet`. The first-write-wins discipline is deliberate: an admin-mediated binding would re-introduce governance authority over the schema-validation surface; a permissionless binding without first-write-wins would let any address overwrite an existing binding and break in-flight attestations. The combination is the only design that satisfies both no-admin and no-overwrite.

Remark 4.1 (Why first-write-wins is structurally necessary). Consider the alternatives. *Admin-mediated binding*: an admin can rotate validators, but admin authority over the validation surface re-introduces the discretionary actor the no-escape-hatches discipline rules out. *Mutable bindings*: any address can rewrite a binding, breaking existing attestations under that schemaId. *Time-locked bindings*: a delay buffers the overwrite, but any non-zero time creates a window in which the binding is uncertain. The first-write-wins rule produces a binding that is both permissionless and stable: anyone can set it once; no one can change it after.

4.4 The atomic-bind pattern

The first-write-wins rule produces a front-running window between `SchemaRegistry.registerSchema` and `AttestationCoordinator.setValidator`. An adversary observing a pending registration can race to set a malicious validator under the new schemaId before the legitimate validator binds. The malicious validator self-attests as bound to the schemaId (satisfying the binding-integrity check) and then accepts content the legitimate validator would have rejected.

The mitigation is atomic registration: both writes execute in a single transaction. The 17 reference `figaro-*` validators are bound this way at genesis (the 18th catalogue schema, `figaro-topology-v1`, is manifest-only and has no on-chain validator), with `script/Deploy.s.sol:_deployAndI` composing the calls in a single deployment broadcast session controlled by one signer (no third-party interleaving against the deployer's nonce stream). For third-party schemas registered post-deploy, the `SchemaRegistrationHelper` contract supplies genuine single-transaction atomicity as a stateless, no-admin, no-fee public helper:

```
function registerSchemaAndValidator(
  bytes32 schemaId,
  uint64 version,
  bytes32 uriHash,
  address validator
) external;
```

The helper composes the two underlying public calls (`SchemaRegistry.registerSchema` and `AttestationCoordinator.setValidator`) atomically. The helper has no authority over either underlying contract; it has no admin, no fee, and no privilege. It is a permissionless composer that makes the two-step pattern safe.

4.5 The 18-schema catalogue

The protocol currently ships eighteen schemas, treated as a coordinated reference set rather than as independent extensions.

1. `figaro-topology-v1` (manifest-only clause; no runtime validator)

2. `figaro-handoff-v1` (physical-exchange mode)
3. `figaro-commerce-v1` (currency, payment, line items)
4. `figaro-geo-v1` (origin / destination geohash)
5. `figaro-fulfilment-v1` (fulfilment method)
6. `figaro-ghg-protocol-v1` (GHG Protocol Corporate Standard; Category-2 disclosure)
7. `figaro-ghg-iso-14064-v1` (ISO 14064 family; Category-2 disclosure)
8. `figaro-ghg-pas-2050-v1` (PAS 2050 product carbon footprint; Category-2 disclosure)
9. `figaro-ghg-en-16258-v1` (EN 16258 transport-emissions methodology; Category-2 disclosure)
10. `figaro-ghg-custom-v1` (custom / non-standard GHG methodology; Category-2 disclosure)
11. `figaro-ghg-measurement-v1` (runtime grams CO2e; Category-1)
12. `figaro-courier-process-v1` (5-stage progression + evidence URI)
13. `figaro-proximity-policy-v1` (committed detection band; Category-2)
14. `figaro-proximity-proof-v1` (per-handoff signed witness; Category-1)
15. `figaro-merchant-process-v1` (merchant per-role event enum)
16. `figaro-courier-process-v1` (courier per-role event enum)
17. `figaro-jurisdiction-v1` (off-chain forum / law / language)
18. `figaro-consent-v1` (cryptographic acceptance of an off-chain document)

The catalogue exhibits several patterns worth flagging. The *sister-schema pattern* (the five GHG schemas, the proximity policy/proof pair, the merchant/courier process schemas) supplies multiple specialized variants of a shared coordination concept; content shape is shared, schemaId is per-standard, and per-standard extensions can be added to a single sister without affecting siblings. The *committed-vs-runtime split* (proximity policy/proof, GHG protocol/measurement) separates the band the parties commit at agreement signing (Category-2, byte-equality enforced) from the runtime witness data filed during execution (Category-1, fresh per attestation). The *manifest-only clause* pattern (`figaro-topology-v1`) represents a shared-vocabulary anchor whose enforcement is purely off-chain; parties commit to the topology at contract-signing time, the indexer reconstructs the DAG from event logs, and no runtime attestation fires. Each pattern is a discipline-level choice the catalogue's authors made deliberately, and each is reusable by future schemas in the family.

Class A and Class B records. A useful evidentiary taxonomy distinguishes records the kernel produces by construction from records participants attest under schema discipline. *Class A* records are kernel byproducts (commitment digest, bond deposits, cumulative-value accumulator state, order status transitions, resolution events): emitted by the kernel, schema-typed at the kernel level, discretion-free, and cryptographically authenticated. *Class B* records are discretionary schema-validated attestations (delivery confirmations, GHG disclosures, handoff attestations, custody-of-seal events, lifecycle stage transitions): produced by participants through

the `AttestationCoordinator`, schema-typed at the *protocol* level, validator-gated, and substantively contestable on truth-of-content grounds. The schema-design discipline of this paper produces Class B records: Layer A specifies the content shape, Layer C enforces the shape at the on-chain validator, the runtime tier renders and submits, and the resulting attestation enters the evidentiary record as Class B input.

4.6 What schema design is not

Three classes of would-be extension fall outside the discipline:

Per-instance payload schemas that try to make every order-specific data shape into a registered schema. Most order-specific data does not need a registered schema; it needs an order-specific encoder and an off-chain decoder. The discipline applies the decision rule (Theorem 3.2) at the point of asking "does this need stable shared interpretation across parties or tools?" If the answer is no, the data is a payload, not a schema candidate.

Schemas with mutable freeform text or large on-chain payloads (typically larger than 1KB). Mutable freeform text violates append-only identity. Large payloads violate the minimal-anchor surface. Both move the schema in the wrong direction.

Schemas with admin, pause, or upgrade hooks. The no-escape-hatches discipline applies to schemas as it applies to the kernel. A schema's validator is pure, immutable, and permissionless once bound; an admin-controlled validator re-introduces the discretionary actor.

5 The Coordinator Pattern

We turn now to the second discipline question: under what conditions does an external mechanism contract compose with the kernel without breaking the bonding equilibrium? The discipline that answers this is the *coordinator pattern*: a set of sufficient conditions on an extension contract's read/write profile against kernel state under which the bonding equilibrium is preserved by composition. We state the conditions formally with composition semantics and prove that they are sufficient.

5.1 Composition semantics

Let $\mathcal{K}$ denote the FIGARO kernel as a state machine with state space $\mathcal{S}_\mathcal{K}$ (the values of the three storage mappings: `processes`, `orderStatus`, `orderProcessId`) and transition function $\delta_\mathcal{K}$ restricted to the two operations `commit` and `resolveProcess`. Let $\mathcal{M}$ denote an external mechanism contract with state space $\mathcal{S}_\mathcal{M}$ and transition function $\delta_\mathcal{M}$.

Definition 5.1 (Composition)**.** We extend the state space to include the kernel contract's ERC-20 token-balance position $\beta_\mathcal{K}$ and the event log $E_\mathcal{K}$ alongside $\mathcal{S}_\mathcal{K}$ proper. The composition $\mathcal{K} \otimes \mathcal{M}$ is the joint state machine over the extended state space in which $\mathcal{M}$ may read from $\mathcal{S}_\mathcal{K}$, $\beta_\mathcal{K}$, and $E_\mathcal{K}$ but may not write to any of them, and $\mathcal{K}$ neither reads from nor writes to $\mathcal{S}_\mathcal{M}$. The composition's transition function admits interleaving via the EVM call graph (an $\mathcal{M}$ transition's call frame may invoke $\mathcal{K}$'s public methods, including state-changing methods like `commit`, but only subject to condition (i) of Theorem 5.2 below: any such invocation either fails authorization or executes as the calling party would have executed it directly through $\mathcal{K}$'s public surface, without producing $\mathcal{M}$-mediated state mutation that the calling party did not authorize).

The asymmetric read/write structure is the load-bearing constraint. $\mathcal{M}$ may consult kernel state to make its decisions, but $\mathcal{M}$ cannot modify kernel state. $\mathcal{K}$ does not need to know about $\mathcal{M}$ at all; the kernel is composition-blind. The extended state space is required to make the composition definition match the operational semantics of EVM contracts: invariants like

progressive collateralization involve $\beta_{\mathcal{K}}$ (the kernel's escrow balance) as well as $\mathcal{S}_{\mathcal{K}}$, and the immutable-evidence invariant is over $E_{\mathcal{K}}$ rather than purely over state mappings.

5.2 The coordinator pattern, formally

Proposition 5.2 (Equilibrium-preserving composition)**.** *The composition $\mathcal{K} \otimes \mathcal{M}$ preserves the kernel's bonding equilibrium if $\mathcal{M}$ satisfies all four of the following conditions:*

(i) Read-only on kernel state. *For every operation $\mathcal{M}.f$ and every kernel-state predicate P that is invariant under $\delta_{\mathcal{K}}$, P is invariant under $\delta_{\mathcal{M}}$ as well: that is, no $\mathcal{M}$ operation can mutate kernel state. Formally: for every $\mathcal{M}.f$, the call frame on which $\mathcal{M}.f$ executes does not invoke any kernel state-changing operation (`commit` or `resolveProcess`) on $\mathcal{K}$ in a way that produces unauthorized state mutation.*

(ii) No alternative settlement path. *$\mathcal{M}$ does not provide an operation that produces value flows equivalent to those of `resolveProcess` but bypasses `resolveProcess` or modifies its preconditions.*

(iii) No discretionary lock-bypass. *$\mathcal{M}$ does not provide an operation that releases bonded capital from $\mathcal{K}$'s escrow under conditions different from those `resolveProcess` enforces.*

(iv) Validator-gated content (where applicable). *If $\mathcal{M}$ accepts content typed by a registered schemaId, $\mathcal{M}$ routes the content through the registered `ISchemaValidator` before accepting it; content typed by a schemaId without a registered validator is rejected.*

The conditions are stated in the claims they constrain: condition (i) preserves the kernel state predicates the verification stack establishes (status monotonicity, transition correctness, cumulative integrity, etc.); condition (ii) preserves buyer dominance; condition (iii) preserves the no-escape-hatches discipline; condition (iv) preserves the schema-validation surface. A mechanism that satisfies all four can be composed onto the kernel with the kernel's equilibrium guarantees still in force.

Sketch. The kernel's bonding equilibrium rests on the six invariants of Section 2. Each of the four conditions above maps to preservation of a subset of those invariants under composition.

Condition (i) preserves invariants (i)–(v): if $\mathcal{M}$ cannot mutate kernel state, the kernel's existing invariants remain invariants under composition. Formally, if $P : \mathcal{S}_{\mathcal{K}} \to \text{Bool}$ is invariant under $\delta_{\mathcal{K}}$ (formally verified at the kernel tier), and $\delta_{\mathcal{M}}$ does not produce state changes in $\mathcal{S}_{\mathcal{K}}$, then P is invariant under $\delta_{\mathcal{K}\otimes\mathcal{M}}$ since the latter is the disjoint union of $\delta_{\mathcal{K}}$ and $\delta_{\mathcal{M}}$.

Condition (ii) preserves invariant (iii) (buyer dominance) and invariant (iv) (atomic resolution) by ruling out alternative paths that would bypass them. If $\mathcal{M}$ provided an operation producing the value flows of `resolveProcess` without satisfying the buyer-only and all-or-nothing preconditions, the escape-hatch-weakness theorem applies and the equilibrium is broken.

Condition (iii) preserves invariant (vi) (no escape hatches) by ruling out lock-bypass operations.

Condition (iv) preserves the schema-validation surface as a component of the protocol-tier integrity. A mechanism that accepts unvalidated content under a schemaId would create a content surface that downstream consumers cannot trust; the validation gate is what makes schema-typed content protocol-grade rather than mechanism-local.

The conditions are jointly sufficient: a mechanism satisfying all four does not interact with kernel state in any way that could break any kernel invariant or any schema-validation surface property. The conditions are also *necessary in the worst case* (relaxing any one produces a documented anti-pattern): a mechanism that mutates kernel state directly (relax condition i) breaks the verified kernel invariants; a mechanism with an alternative settlement path (relax condition

ii) implements the timeout / arbitrator / governance-override escape-hatch the escape-hatch-weakness theorem forbids; a mechanism with a discretionary lock-bypass (relax condition iii) implements an admin-controlled fund-withdrawal path the no-escape-hatches discipline forbids; a mechanism that accepts unvalidated schema-typed content (relax condition iv) creates a content surface that downstream consumers cannot trust as protocol-grade. We do not claim the conditions are necessary in every model; weaker conditions on a mechanism with sufficiently restricted power (e.g., a pure view contract) might suffice for narrower preservation claims. The four conditions are sufficient for full equilibrium preservation and necessary in the worst-case sense above. □

5.3 The AttestationCoordinator as worked example

The `AttestationCoordinator` contract is the canonical worked example of the coordinator pattern. The contract accepts attestations against bonded orders, gates them through registered `ISchemaValidator` contracts, and emits typed events without ever modifying kernel state. A second admission gate enforces a Merkle-inclusion proof against the order's signed `agreementHash`: only clauses committed at contract-signing time can be attested under, so a clause that was not part of the bilateral agreement cannot land an attestation against the order. The contract's verification surface discharges Theorem 5.2's conditions explicitly.

Conditions discharged. The Certora specification `certora/AttestationCoordinator.spec` contains seven declared CVL rules (eight including the parametric expansion of one rule reported in the cloud verification artifact). Two of them are parametric over every public function on `AttestationCoordinator` and are the direct discharge of condition (i): `attestationCannotChangeOrderStatus` and `attestationCannotChangeProcessState`. Each rule is universally quantified over methods f on the contract: for any public function f and any kernel-state value σ before and after f's execution, σ is unchanged. The Certora prover discharges this universally for the AttestationCoordinator's public surface.

Conditions (ii) and (iii) are discharged by the contract's construction: `AttestationCoordinator` has no operation that transfers tokens from the kernel's escrow, and no operation that calls `resolveProcess` or any kernel state-changing function on the kernel. The contract is read-only on the kernel.

Condition (iv) is discharged by the `noValidatorBlocksBuyerAttestation` CVL rule, which proves that any attestation under a schemaId with no registered validator reverts.

The four conditions thus reduce to verification-stack requirements that a candidate coordinator can be checked against mechanically. The coordinator pattern is not a guideline to be followed at the developer's discretion; it is a contract-level discipline whose conditions are dischargeable with the existing verification toolchain.

5.4 What the pattern is not

The pattern is not a generic external-call disclaimer. A mechanism that calls into the kernel's view functions, reads state, and returns derived values is composition-safe; a mechanism that calls into the kernel's state-changing functions on behalf of a third party may or may not be composition-safe, depending on whether the third-party authorization is properly delegated. The `IRoleResolver` interface supplies the authorization-delegation pattern for the latter case (a role-based attestation routed through a mechanism that consults the role-resolver to authorize the caller).

The pattern is also not a substitute for the schema-validation discipline. A mechanism that accepts schema-typed content must route it through the registered validator; a mechanism that produces schema-typed events must produce content the registered validator would have

accepted. The two disciplines (coordinator pattern and schema design) are independent and both must be satisfied for an extension to be protocol-grade.

6 Runtime Composition

We turn now to the runtime tier: the layer above the protocol where institutional shapes are composed and rendered. The runtime tier is not part of the protocol's verification surface (it does not affect kernel state and does not change the equilibrium), but it is part of the protocol's deployment surface (without a runtime, the protocol has no users, only contracts). The discipline at the runtime tier is software architecture: how to compose institutional shapes from reusable parts without re-implementing the protocol per deployment.

6.1 Seven-layer composition pipeline

The runtime composes institutional surfaces through seven layers.

1. **Protocol truth.** Contract reads, event subscriptions, and the cumulative state derived from them. This layer is authoritative; nothing above it can override it.

2. **Semantic derivation.** The derivation of role contexts, capability bindings, mechanism trust boundaries, and guarantee/risk objects from protocol state. This layer is also authoritative: a runtime that renders a role the semantic layer has not derived is rendering a role that is not in the protocol.

3. **Institution assembly.** The structural declaration of an institutional shape: which roles compose, which mechanisms are bound, which views are exposed. The assembly is the runtime's canonical document.

4. **Mechanism packages.** Reusable units that own a mechanism's contract bindings, semantic adapters, capability mappings, default modules, and guarantee/risk copy. A mechanism package is the unit the runtime composes; an assembly references packages rather than individual modules.

5. **Service bindings.** Off-chain or hybrid infrastructure bindings: identity resolution, catalogue metadata, discovery and search, messaging and handoff, evidence transport, geospatial filtering. The bindings are resolved through stable interfaces, not hardwired per archetype.

6. **View definitions.** Per-surface UI composition: route or surface id, accepted context, visible slots, module ordering, role-specific visibility.

7. **Skin / branding fields.** Presentation-only personalization: theme tokens, imagery, type and spacing preferences, non-semantic copy. In the current implementation these are lightweight fields attached to the assembly binding (accent colour, logo, hero, theme class) rather than a heavyweight separate pipeline stage; we list them as a distinct layer because they sit at a distinct trust boundary (presentation-only, never semantic), not because they require their own composition machinery. Skin / branding fields cannot alter capability validity, mechanism authority, risk boundaries, or settlement semantics.

The pipeline is layered: each layer consumes the layers below it and produces output the layers above it consume. The runtime renders an institution by walking the pipeline from protocol truth at the bottom to skin bundle at the top. The structural property the discipline enforces is that institution-specific mutation lives in layers 3 through 7; the bottom two layers are runtime authority and not institution-overridable.

6.2 Assembly registry pattern

An institution assembly is a JSON document declaring the institutional shape: roles, mechanism packages, view definitions, module placements, narrative defaults. The assembly is the runtime's structural primary; the rest of the pipeline is derived from it. The assembly registry pattern handles four operations:

- *Discovery*: which assemblies exist for a given archetype, role, or geographic region.
- *Parsing*: validating the assembly's JSON against the runtime's schema for institutional shape.
- *Validation*: cross-checking that the mechanism packages referenced exist, that the view definitions reference valid modules, and that the role mappings are coherent.
- *Publication*: making a new assembly available to the runtime, with the assembly's content addressed for interoperability.

The pattern is implemented in `frontend/lib/shared/assemblyRegistry.ts` (discovery and indexing), `frontend/lib/shared/assemblyParser.ts` (parsing and validation), and `frontend/lib/shared/as` (publication to the runtime).

The current reference set comprises six assemblies: `local-commerce` (the canonical food-delivery archetype), `direct-sale` (a buyer-to-merchant single-leg arrangement), `figaro-equipment-rental`, `figaro-freelance`, `figaro-procurement`, and `figaro-disclosure-review`. Each is a worked example of the assembly format and an operational template that can be specialized into a deployed institution.

6.3 Module system

Modules are the unit the assembly's view definitions place in the rendered surface. A module owns a UI component, a set of typed actions it can dispatch, and the semantic context it consumes. The module registry (`frontend/components/modules/`) catalogues the modules currently shipped: catalogue editing, cart composition, order creation, attestation submission, delivery coordination, GHG disclosure, FIG token operations, and others.

The module system has a structural property worth flagging: a module is composition-safe if and only if its dispatched actions go through the typed action model (rather than producing direct contract calls) and its semantic context comes from the semantic derivation layer (rather than from local state hardcoded into the module). The discipline matches the coordinator pattern's separation of concerns at the runtime tier: the protocol-tier coordinator pattern says external mechanisms read kernel state and emit events without writing to the kernel; the runtime-tier module discipline says modules read semantic state and dispatch typed actions without writing to the runtime's authoritative layers.

6.4 The lens system

A particular runtime-tier construct worth treating separately is the *lens* pattern: a typed context object that a module receives and that exposes a constrained subset of the runtime's full state. A buyer-side module receives the buyer's lens onto the process; a seller-side module receives the seller's lens; an auditor's module receives the auditor's lens. Each lens exposes only what its role context licenses; the module cannot reach beyond the lens to access other roles' state.

The lens pattern is the runtime's enforcement of the role-segregation that the kernel enforces at the bonding-and-resolution surface. The kernel's role-segregation is mathematical (only the buyer can call `resolveProcess`); the runtime's role-segregation is operational (only a buyer-context module can dispatch buyer-side actions). The two reinforce each other: a runtime that

exposed seller-side state to a buyer-context module would let the buyer submit attestations as the seller, which the kernel would reject at signature verification but which the runtime should not have attempted in the first place.

6.5 The designer canvas

A specialized runtime surface deserves separate treatment: the assembly designer canvas (`ProcessGraphCanvas` + `AgreementDrawer`). The canvas is a composition surface where assembly authors fork an existing reference assembly or start from blank, modify the bonded-process graph, bind schemas to per-node clauses through a side drawer, and save drafts to local storage.

The canvas is not a wizard, not a recommendation engine, not a generative-AI interface; it is a structural editor for assemblies. The designer pulls the runtime's primitives (commerce-v1 commitments, handoff-v1 attestations, fulfilment-v1 attestations, etc.) onto the canvas as draggable elements; the author composes them into the shape the institutional context requires; the resulting assembly is saved as a draft.

The discipline the canvas enforces is structural: an assembly that violates the runtime's composition rules cannot be saved. A fan-out node where one fan-out lacks a corresponding mechanism package is flagged; an attestation node referencing an unbound schemaId is flagged; structural composition failures (mismatched mechanism packages, dangling sub-orders, topology shapes the runtime does not admit) are surfaced before the draft can be persisted. The canvas surfaces the discipline rather than hiding it; an author who saves an assembly through the canvas has by construction produced one that satisfies the runtime's structural integrity.

6.6 The (marketing) / (app) split

A practical runtime-architecture choice worth naming: the frontend separates routes into two top-level groups, `frontend/app/(marketing)/` for publication-and-explanation surfaces and `frontend/app/(app)/` for transactional surfaces. The split has three structural properties.

No wallet provider in marketing routes. A user who has never connected a wallet must be able to read every marketing route. The `(marketing)` group does not load the wallet provider; the `(app)` group does.

Wallet-connect is signing prerequisite, not login. `(app)` routes mount the wallet provider but do not gate read-only content behind connection state. Inline write affordances use the canonical `WalletGate` wrapper to require connection at the moment of signing rather than at route entry.

Two distinct headers. The marketing header carries marketing nav (Discover, About, Cryptoeconomics, etc.); the (app) header carries protocol-surface nav (Connect Wallet, Inbox, Orders, etc.). The two are not collapsed.

The split's structural property is that a reader of the publication content (the eight Zargham-discipline papers, the `cryptoeconomics` page, the `groups` page, etc.) is not asked to connect a wallet to read; a participant's transactional surface is wallet-gated by structural design rather than by per-page logic. The architectural separation matches the trust-tier separation of the underlying protocol: marketing is untrusted publication, (app) is signing-required interaction, and the route architecture reflects the trust layers rather than collapsing them into one shell.

7 What the Discipline Refuses

The discipline is more visible in what it refuses than in what it admits. We catalogue the recurring failure modes the discipline prevents.

Kernel changes. The kernel is frozen. Any proposed extension that requires a kernel change is, by the discipline, not a candidate extension; it is a proposal for a different protocol. The

escape-hatch-weakness theorem makes this not a stylistic preference but a mathematical necessity: any unilateral exit path from the bonded state weakens the Nash equilibrium. The protocol's narrowness is its security property.

Admin paths. Any proposed extension that introduces admin authority over schema binding, validator selection, mechanism activation, or kernel state is refused. The first-write-wins binding pattern is the specific implementation of the no-admin discipline at the schema-validation surface; analogous patterns apply elsewhere (e.g., the `OperatorRegistry`'s permissionless registration without admin gating).

Force-majeure escape clauses. Any proposed extension that lets a participant avoid bond loss in external events (weather, regulatory action, force majeure) is refused. The structural answer to force-majeure-style concerns is composition with parallel insurance processes: the participant buys insurance from an external insurer naming themselves as beneficiary, and the insurance settlement is independent of the bonded commitment. Bonded settlement does not contemplate external-event-conditional release; insurance does. The two compose; neither replaces the other.

Multi-currency cross-leg coordination. Any proposed extension that lets a process chain denominate different legs in different currencies is refused. Multi-currency coordination requires an external rate of substitution (oracle, DEX, pre-agreed FX rate) that re-introduces a discretionary actor and breaks the same-unit comparability on which the 2× Nash-stability result rests. Multi-currency coordination at the participant level is achieved through wallet-side swaps before commitment; the kernel sees one currency per process.

Schemas with mutable freeform text. Any proposed schema with large-text or mutable-text fields is refused. The minimal-anchor surface and append-only identity together rule out anchoring substantial mutable content on chain. Such content belongs off-chain, content-addressed by hash with the hash registered as the schema's reference.

Modules with direct contract calls. Any runtime-tier module that bypasses the typed action model and produces direct contract calls is refused. The action model's discipline is what makes the runtime's compositional safety analyzable; a module that escapes the discipline can produce state changes the runtime's other layers do not know about, which breaks the composition pipeline.

Skin bundles that alter semantics. Any skin bundle that affects capability validity, mechanism authority, risk boundaries, or settlement semantics is refused. A skin is presentation-only; an institution that wants different semantics writes a different assembly, not a different skin.

Soulbound reputation as protocol primitive. Any proposed extension that introduces a non-transferable reputation token bound to a wallet at the protocol layer is refused. Reputation is a graph-tier signal, not a kernel-tier construct; introducing soulbound reputation at the protocol layer reifies what is appropriately a graph-level pattern (settlement history is already on chain and readable as reputation by any party who wants to read it that way) into a privileged-credential construct that weakens the kernel's actor-neutrality property.

Fee discounts conditioned on protocol-tier signals. Any proposed extension that conditions kernel-tier or protocol-tier behavior on a fee discount, a green-bond adjustment, or any other rate-modifying signal is refused. The 2× bonding ratio is the minimum viable equilibrium

under the minimum-multiplier result; introducing a fee-discount mechanism that modifies the effective ratio for some class of participant breaks the equilibrium for that class. Whatever incentive a fee discount would supply belongs at the assembly or operator-registry tier as a parameter the parties choose, not at the bonding rule.

8 Conclusion

Above the kernel sits a protocol layer (extensions to the kernel under the discipline of Sections 3 to 5) and a runtime layer (institutional shapes composed under the discipline of Section 6). Both layers are research objects in computer-science register, and both have a discipline derived from the kernel's security requirements rather than from convenience or developer ergonomics.

The protocol-extension doctrine produces a decision rule (Theorem 3.2) for whether a proposed feature belongs in the protocol; schema design as a CS discipline produces a four-layer verification stack with append-only identity and first-write-wins binding; the coordinator pattern produces formal sufficient conditions (Theorem 5.2) for equilibrium-preserving composition. Each is a contract the discipline holds extension authors to; together they make extension to the kernel a rigorous engineering practice rather than an opinion-driven one.

The runtime-tier composition produces a seven-layer pipeline (from protocol truth to skin bundle), an assembly registry pattern, a module system constrained by the typed action model, a lens system enforcing role-context separation, a designer canvas surfacing discipline at authoring time, and a route-architecture split between marketing and transactional surfaces. The runtime tier is not verified the way the protocol tier is, but its discipline is operationally checkable, and the open question is whether formal verification at the runtime tier is worth the work.

The contribution is the discipline itself, made explicit and research-grade. The present paper supplies the connecting tissue: how the kernel becomes a protocol becomes a runtime, with the discipline at each layer specified well enough that an extension author or runtime architect can apply it.

Acknowledgments

The protocol-extension and runtime-composition work developed incrementally alongside the kernel. The extension doctrine document originated as an architectural note on GHG schema anchoring and generalized; the runtime model emerged from operational requirements of the local-commerce reference archetype and generalized. The COALA collaborators have been a consistent interlocutor on the legal-tier interactions of the protocol-extension and schema-validation surfaces.

References

Giordano, F., Lockyer, M., and Castonguay, P. EIP-1271: Standard Signature Validation Method for Contracts. Ethereum Improvement Proposal 1271, July 2018. `https://eips.ethereum.org/EIPS/eip-1271`

Joyce, R. and others. EIP-712: Ethereum Typed Structured Data Hashing and Signing. Ethereum Improvement Proposal, September 2017.

Solidity Team. *Solidity Documentation*, version 0.8.x series. Ongoing as of 2024.

OpenZeppelin. *OpenZeppelin Contracts: Reference Implementations for Solidity Patterns*. OpenZeppelin Contracts Documentation, ongoing as of 2024.

Certora. *Certora Verification Language (CVL): Documentation and Reference*. Certora Technical Documentation, ongoing as of 2024.

Foundry. *Foundry: Ethereum Smart Contract Testing Framework*. Foundry Book, ongoing as of 2024.